

Docker Image Signing & Verification Handson

Docker Content Trust (DCT) uses Notary v1 under the hood to sign and verify images.

When DCT is enabled, docker push signs the image, and docker pull verifies its signature before allowing the image.

1. Enable Docker Content Trust : `export DOCKER_CONTENT_TRUST=1`
This enables signing & verification for all Docker commands in this shell session.
You can also disable it temporarily by: `export DOCKER_CONTENT_TRUST=0`
2. Login to Docker Hub : `docker login` (Use your dockerhub username and password to login) or create & use a custom Personal access token PAT.

```
(base) labuser@ip-172-31-30-223:~$ export DOCKER_CONTENT_TRUST=1
(base) labuser@ip-172-31-30-223:~$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you do
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT
s-tokens/

Username: saurabhnegi2306
Password:
WARNING! Your password will be stored unencrypted in /home/labuser/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

Login Succeeded
(base) labuser@ip-172-31-30-223:~$ _
```

3. Create a test Dockerfile :

```
FROM alpine:3.19
```

```
CMD ["echo", "Hello from signed image!"]
```

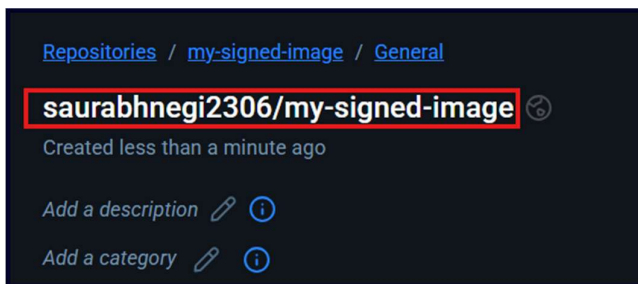
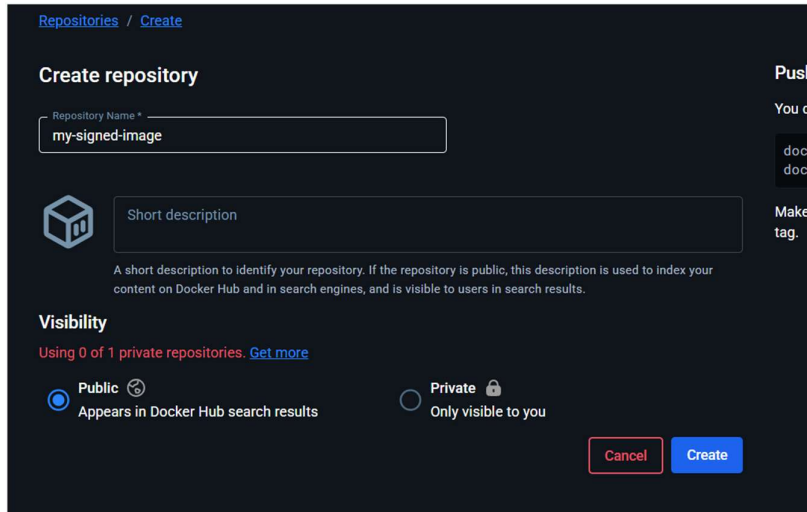
4. Build the image locally : `docker build -t my-signed-image:latest .`

```
(base) labuser@ip-172-31-30-223:~$ docker build -t my-signed-image:latest .
[+] Building 3.9s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 96B
=> [internal] load metadata for docker.io/library/alpine:3.19
=> [auth] library/alpine:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/1] FROM docker.io/library/alpine:3.19@sha256:3be987e6cde1d07e873c012bf6cfe941e6e85d16ca5fc9
=> => resolve docker.io/library/alpine:3.19@sha256:3be987e6cde1d07e873c012bf6cfe941e6e85d16ca5fc9
=> => sha256:3be987e6cde1d07e873c012bf6cfe941e6e85d16ca5fc9b8bedc675451d2de67 8.08kB / 8.08kB
=> => sha256:65d86f451d12fb1de9db57a9226a899d80a0897cf0f1645faa565be3a268e621 1.02kB / 1.02kB
=> => sha256:f457758f54f724a146f1bd2dc50d4ee00cd93cc0102a18723969ed9bee43a2c9 597B / 597B
=> => sha256:07e9a47f0c334cceba1b05e86ef0150c84564a9b9e9d4ae9dc9a5ebc85ef2b7c 3.35MB / 3.35MB
=> => extracting sha256:07e9a47f0c334cceba1b05e86ef0150c84564a9b9e9d4ae9dc9a5ebc85ef2b7c
=> exporting to image
=> => exporting layers
=> => writing image sha256:26a7028f68d0b9eafd1e7ef0deddf678beb3d4213ee98958dfbb8581d1e75db3
=> => naming to docker.io/library/my-signed-image:latest
(base) labuser@ip-172-31-30-223:~$ _
```

Check the image : `docker image ls`

```
(base) labuser@ip-172-31-30-223:~$ docker image ls
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
my-signed-image      latest      26a7028f68d0 4 weeks ago   7.73MB
(base) labuser@ip-172-31-30-223:~$
```

5. Login to the docker hub GUI and create a public repository and note down the full path of the repository which should be <<USERNAME>>/<<REPONAME>>. We will need to tag our local images with this name format so that those can be pushed into this repo.



6. Tag previously created Image : `docker tag my-signed-image:latest saurabhnegi2306/my-signed-image:latest`

```
(base) labuser@ip-172-31-30-223:~$ docker tag my-signed-image:latest saurabhnegi2306/my-signed-image:latest
(base) labuser@ip-172-31-30-223:~$ docker image ls
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
my-signed-image      latest     26a7028f68d0  4 weeks ago   7.73MB
saurabhnegi2306/my-signed-image  latest     26a7028f68d0  4 weeks ago   7.73MB
(base) labuser@ip-172-31-30-223:~$ _
```

7. Push the latest tagged image to your docker hub repo :

`docker push saurabhnegi2306/my-signed-image:latest` : Docker will generate a **root key** and a **repository key** if this is your first time signing. You'll be prompted to set **passphrases** for the keys. The keys are stored locally in `~/.docker/trust/`.

```
(base) labuser@ip-172-31-30-223:~$ docker push saurabhnegi2306/my-signed-image:latest
The push refers to repository [docker.io/saurabhnegi2306/my-signed-image]
81b3f4a69684: Mounted from library/alpine
latest: digest: sha256:8d809f8eb650f206196b94b1a99f41ff53a604ed922e5f0b0c40985ff07e1d95 size: 527
Signing and pushing trust metadata
You are about to create a new root signing key passphrase. This passphrase
will be used to protect the most sensitive key in your signing system. Please
choose a long, complex passphrase and be careful to keep the password and the
key file itself secure and backed up. It is highly recommended that you use a
password manager to generate the passphrase and keep it safe. There will be no
way to recover this key. You can find the key in your config directory.
Enter passphrase for new root key with ID d646139:
Passphrase is too short. Please use a password manager to generate and store a good random passphrase.
Enter passphrase for new root key with ID d646139:
Repeat passphrase for new root key with ID d646139:
Enter passphrase for new repository key with ID a3eb5b4:
Repeat passphrase for new repository key with ID a3eb5b4:
Finished initializing "docker.io/saurabhnegi2306/my-signed-image"
Successfully signed docker.io/saurabhnegi2306/my-signed-image:latest
(base) labuser@ip-172-31-30-223:~$ _
```

Root Key

- **Purpose:** The highest level key in Docker Content Trust.
- It signs the **delegation keys** (including the repository key).
- Think of it as the **master key** for your trust system.
- It is created **once per Docker Hub account**, not per repository.
- **Stored locally only** (usually in ~/.docker/trust/private/). You should **never share it**.
If someone steals your root key, they could replace repository keys and take over your entire account's trust metadata.

Repository Key

- **Purpose:** Signs tags (e.g., :1.0, :2.0) within a **specific repository**.
- One repository key per repo (e.g., myuser/signed-demo).
- It allows signing image versions without needing to use the root key every time.

If someone steals your repo key, they could tamper with images in that repo, but not other repos.

NOTE: Best practice is to backup these keys securely to avoid issues in case of these keys are deleted/lost.

8. Now we will simulate what would happen if we try pull and unsigned image from docker hub.

Disable signing temporarily using : export DOCKER_CONTENT_TRUST=0

Tag the image differently with signing disabled : docker tag saurabhnegi2306/my-signed-image:latest saurabhnegi2306/my-signed-image:test

```
(base) labuser@ip-172-31-30-223:~$ export DOCKER_CONTENT_TRUST=0
(base) labuser@ip-172-31-30-223:~$ docker tag saurabhnegi2306/my-signed-image:latest saurabhnegi2306/my-signed-image:test
(base) labuser@ip-172-31-30-223:~$ docker push saurabhnegi2306/my-signed-image:test
The push refers to repository [docker.io/saurabhnegi2306/my-signed-image]
81b3f4a69684: Layer already exists
test: digest: sha256:8d809f8eb650f206196b94b1a99f41ff53a604ed922e5f0b0c40985ff07e1d95 size: 527
(base) labuser@ip-172-31-30-223:~$
```

9. Now enable DCT again: export DOCKER_CONTENT_TRUST=1
and now try to pull image saurabhnegi2306/my-signed-image:test which was pushed without signing.

```
(base) labuser@ip-172-31-30-223:~$ export DOCKER_CONTENT_TRUST=1
(base) labuser@ip-172-31-30-223:~$ docker pull saurabhnegi2306/my-signed-image:test
No valid trust data for test
(base) labuser@ip-172-31-30-223:~$
```

This simulates someone uploading an unsigned/tampered image → DCT prevents pulling it. So if someone gets access to your dockerhub account but doesn't have the root or repo key for the repository, he/she won't be able to upload a signed image and if users have DCT enabled, they will see an error while pulling an image which was uploaded by an unauthorized person.

- With DCT enabled, Docker **only pulls images that have valid signatures**.
- Any tampered or unsigned image will be rejected.

This is how DCT protects against supply chain attacks in the pull/push process.

Note : if repo keys are lost but you still have ROOT key, you can create a new repo key and attach to the same repo.